ECE 5242 Project 3: Wheel-odometry-based Mapping Checkup Report

Hugo Onghai[1]

[1]ECE Department at Cornell Tech

Abstract

In this checkup report, we give an overview of what we have accomplished thus far for Project 3 of ECE 5242, Intelligent Autonomous Systems. Project 3 challenges us to implement simultaneous localization and mapping (SLAM) of a four-wheeled, skid-steer robot equipped with four wheel encoders, a LIDAR scanner, and an inertial measurement unit (IMU). Here, we discuss our prelimiary results with wheel-odometry-based mapping. Evaluated empirically, we find good alignment between our predicted occupancy grid and mapping and the provided maps for the training data.

Keywords: SLAM, Dead Reckoning

Correspondence: hpo8@cornell.edu

1. Introduction

Mapping enables robots to understand their environment, and roots perform localization to determine where they are within that environment. These two tasks are crucial for accurate obstacle avoidance, navigation, and pathing.

Humans and other biological creatures can perform mapping and localization subconciously. Eyes and vestibular senses train neuron activity which learn to encode heading directions, object permanency, and spatial awareness. For robots without eyes and ears, we use sensors to take measurements that build our map and location. In our case, LIDAR replaces eyesight, wheel encoders replace "counting steps," and IMU replaces the vestibular senses.

LIDAR stands for light detection and ranging. It is an active sensor, unlike our eyes, that rapidly emits laser beams in all directions. Roughly speaking, these laser beams reflect off of nearby objects or walls and head back towards the LIDAR sensor. Then, the LIDAR sensor measures the time-of-flight of each beam to measure how distant objects are from the sensor.

Wheel encoders count the revolutions of each wheel. Given we know the dimensions of each wheel and the size of the robot chassis, we can estimate the rotation and movement of the robot.

IMU stands for inertial measurement unit. It combines a gyroscope and accelerometer to precisely track angular velocities and accelerations in three dimensions. We can combine this with our wheel encoders to more accurately measure the true heading direction of the robot.

The processes of estimating robot position from estimated heading direction and distance traveled is called dead reckoning. We employ it here.

Alongside dead reckoning, we use the angle and distance data from the LIDAR to map the robot's surroundings. We estimate the position of so-called hit points based on the angle and distance of the interacting object and the robot. This allows us to build a map which we call an occupancy grid.

We expound upon our implementation in Section 3. But first, we emphasize the problem we wish to solve.

2. Problem Statement

Given a robot navigating an unknown indoor environment, construct a map of said environment using the robot's LIDAR, IMU, and wheel encoder time series data.

3. Experimental Approach

In this checkup report, we approach our problem statement from Section 2 with dead reckoning and log-odds occupancy grid mapping.

3.1. Data Loading and Processing

3.1.1. Wheel Encoders and IMU.

First, we load our data with code provided from Jinwook Huh and Travers Rhodes. Their code converts the `.mat` binary data files into Pandas DataFrames. The wheel encoder data comes as a series of ticks captured by the encoder and the timestamp of each tick-count, where 360 ticks is one full rotation of the corresponding wheel. The format of the IMU data is the same as the IMU data in project 2: accelerations and angular velocities along the x, y, and z axes along with the timestamps of measurement.

All these arrays of measurements are then smoothed with the Kalman filter made for Project 2. Then, the front left and rear left wheel encoder ticks are averaged; and the front right and rear right wheel encoders ticks are averaged. This assumes that the wheels on the same side rotate in concert, allowing us to get an average measure of rotations for each side. This is important because skid-steer driving geometries are much simpler when dealing with two wheels.

Next, we convert the encoder data into arch lengths, as discussed in lecture. For our left encoder ticks e_L and our right encoder ticks e_R, we compute the distance each side's wheels spun (s_L and s_R respectively) as

$$s_L = e_L \times \frac{1}{360} \times \pi W_D \tag{1}$$

$$s_R = e_R \times \underbrace{\frac{1}{360}}_{\text{1 revolution per 360 ticks}} \times \pi W_D$$

where W_D represents the diameter of the wheel, taken from the robot spec-sheet.

Then, we calculate the change in heading angle at each timestamp based on the wheel encoders, $\Delta\theta_{\text{enc}}$, as the difference between these arch lengths, divided by the width of the robot chassis. However, as we will discuss later, this width could not be taken directly from the spec-sheet and was inflated for more accurate dead reckoning. Since the wheels slip as the robot turns, augmenting body width corrects for over-estimated arch length distances.

$$\Delta\theta_{\text{enc}} = \frac{s_R - s_L}{\text{Body Width}} \tag{2}$$

We integrate $\Delta\theta_{\text{enc}}$ to find the heading direction over time as determined from the wheel encoders.

Simultaneously, we integrate the IMU's measured angular velocity about the z-axis to find another measurement of the heading direction over time.

$$\int_{t_0}^{t_f} \text{gyro_z}\, dt = \theta_{\text{imu}} \tag{3}$$

We then take a weighted average of these two separate estimations of heading direction to get a stronger estimate. Many weights were tried but we ultimately landed on an equal mixture of each estimate.

3.1.2. LIDAR.

The LIDAR data is provided to in timesteps. For each angle of laser emission, the distance that laser traveled before returning to the LIDAR sensor is stored. Effectively, this is a polar-coordinate representation of the LIDAR data.

We convert these polar angles and distances into cartesian coordinates. For a given angle of laser emission γ, distance of laser travel d, and the current position and heading direction of the robot (x, y, θ) we can find the cartesian coordinates of the measured object (ox, oy) with trigonometry:

$$ox = d\cos(\theta + \gamma) + x \tag{4}$$
$$oy = d\sin(\theta + \gamma) + y$$

At last, we have our estimated locations of objects (ox, oy) which we can use to map out our environment.

3.2. Occupancy Grid

Our (ox, oy) inform our belief of where objects are and are not in our space. Points in (ox, oy) are more likely to have objects, since the LIDAR detected something there. It could have been erroneous or some dust in the air, so we cannot make any guarantees yet. Points between (ox, oy) and the robot's position (x, y) are more likely to not be objects, since the LIDAR would have otherwise detected them. We can use our values of (ox, oy) and (x, y) at every timestamp to reconstruct our occupancy grid map. Moreover, we can use Bresenham's Line algorithm to find gridpoints likely to not be objects.

An occupancy grid discretizes our indoor environment, which makes our map more robust to noise. Each gridpoint stores a real number between zero and one that represents the log-odds of the gridpoint being an object over it being free-space (not an object). We initialize each gridpoint with zero odds of being a wall and increment our log odds from there if there exists a measured (ox, oy) that corresponds to that gridpoint, or we decrement our log odds if its found to be free-space. By how much we increment is a fine-tunable parameter, and we empirically chose to increment by 0.5 and decrement by 0.1.[1]

4. Results and Discussion

Our preliminary implementation was applied to three sets of robot movement and observation data: so-called routes 20, 21, and 23. We had reference maps for routes 20 and 23 to determine our performance.

We visualized our dead reckoning with animations. In Figure 1, some snapshots of our robot's unfolding path are shown below.

[1]This implementation is slightly different from what I presented in class. For class, I had initialized the log-odds as 0.5.

Figure 1 – Snapshots of our dead reckoning of the robot in Train20. The green line is the robot's path, and the red dots depict LIDAR hits at the robot's current position. The final full trajectories of the robot's motion in the map is shown later.

Upon completion of the trajectory, we obtain graphs that depict the full robot's motion within the indoor environment. These are shown below in Figure 2, Figure 3, and Figure 4.

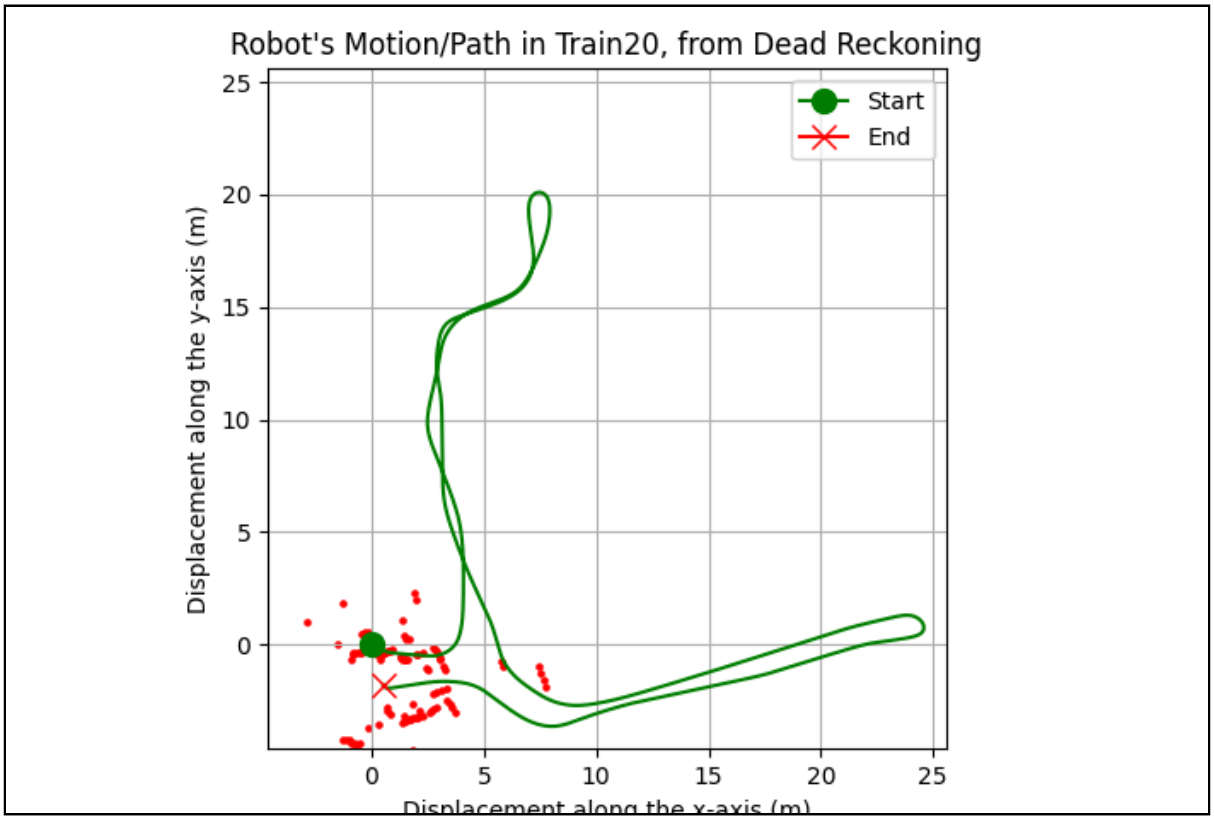

Figure 2 – The final full trajectories of the robot's motion in the route 20.

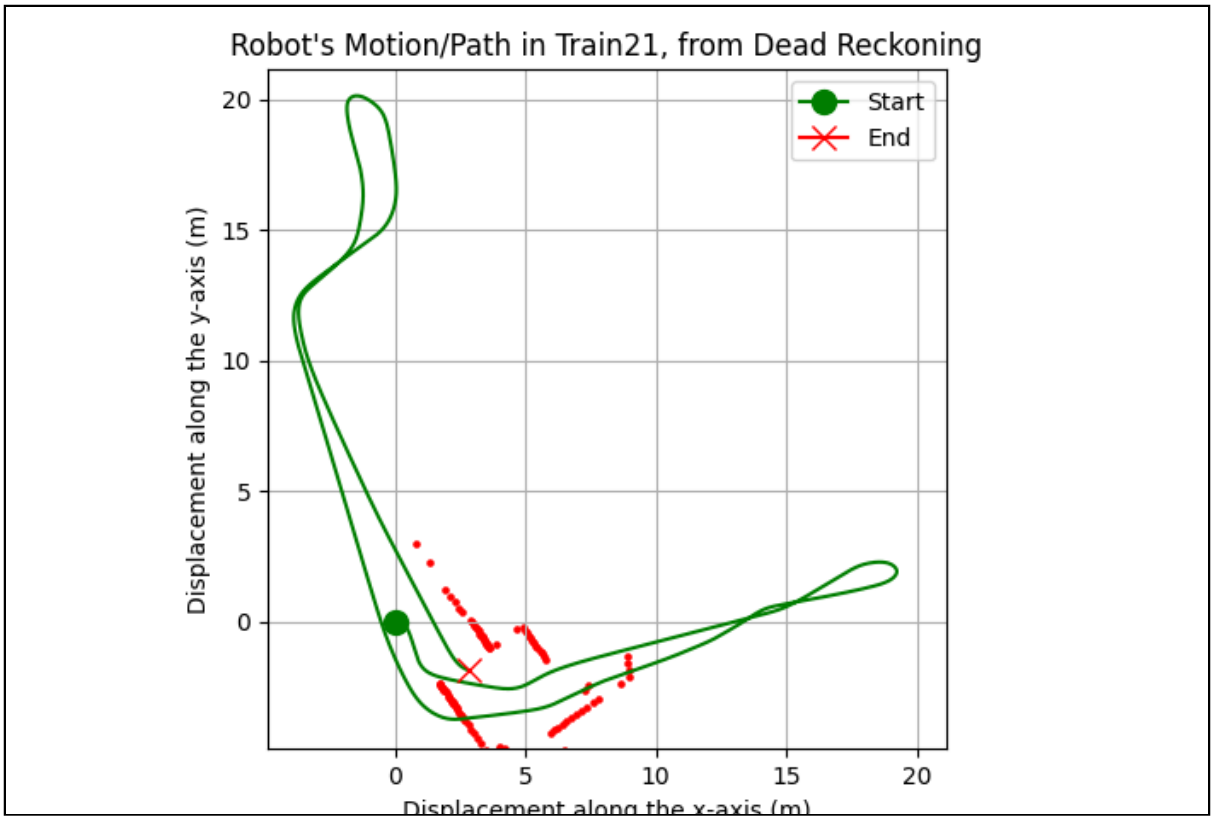

Figure 3 – The final full trajectories of the robot's motion in the route 21.

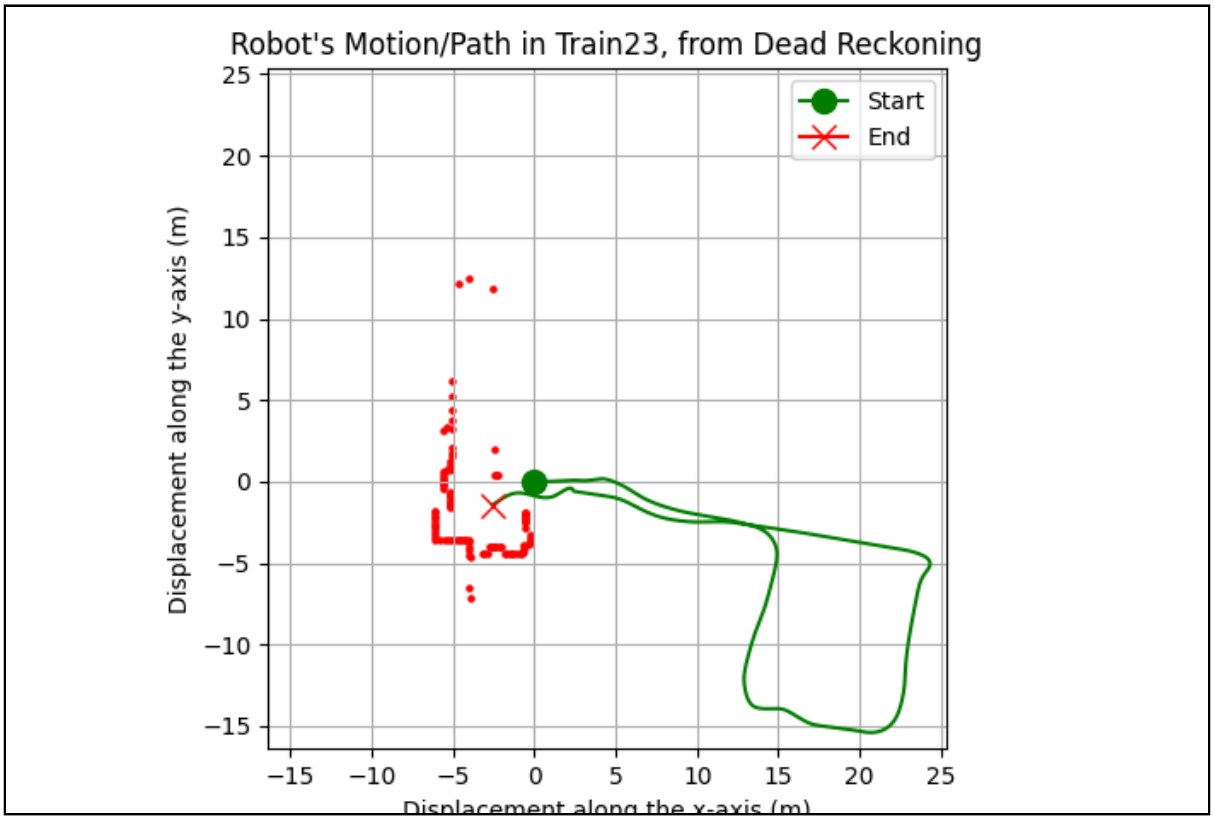

Figure 4 – The final full trajectories of the robot's motion in the route 23.

So far, we've only displayed the LIDAR hits at each current position. We combine the information from all lidar hits to construct our occupancy grids. Recall that lidar hits increase a

gridpoints's log-odds of being an obstacle and colinear points between the hit and the robots position incresae a gridpoint's log-odds of being free space. The occupancy grids for 20, 21, and 23 are shown below in Figure 5, Figure 6, and Figure 7, respectively.

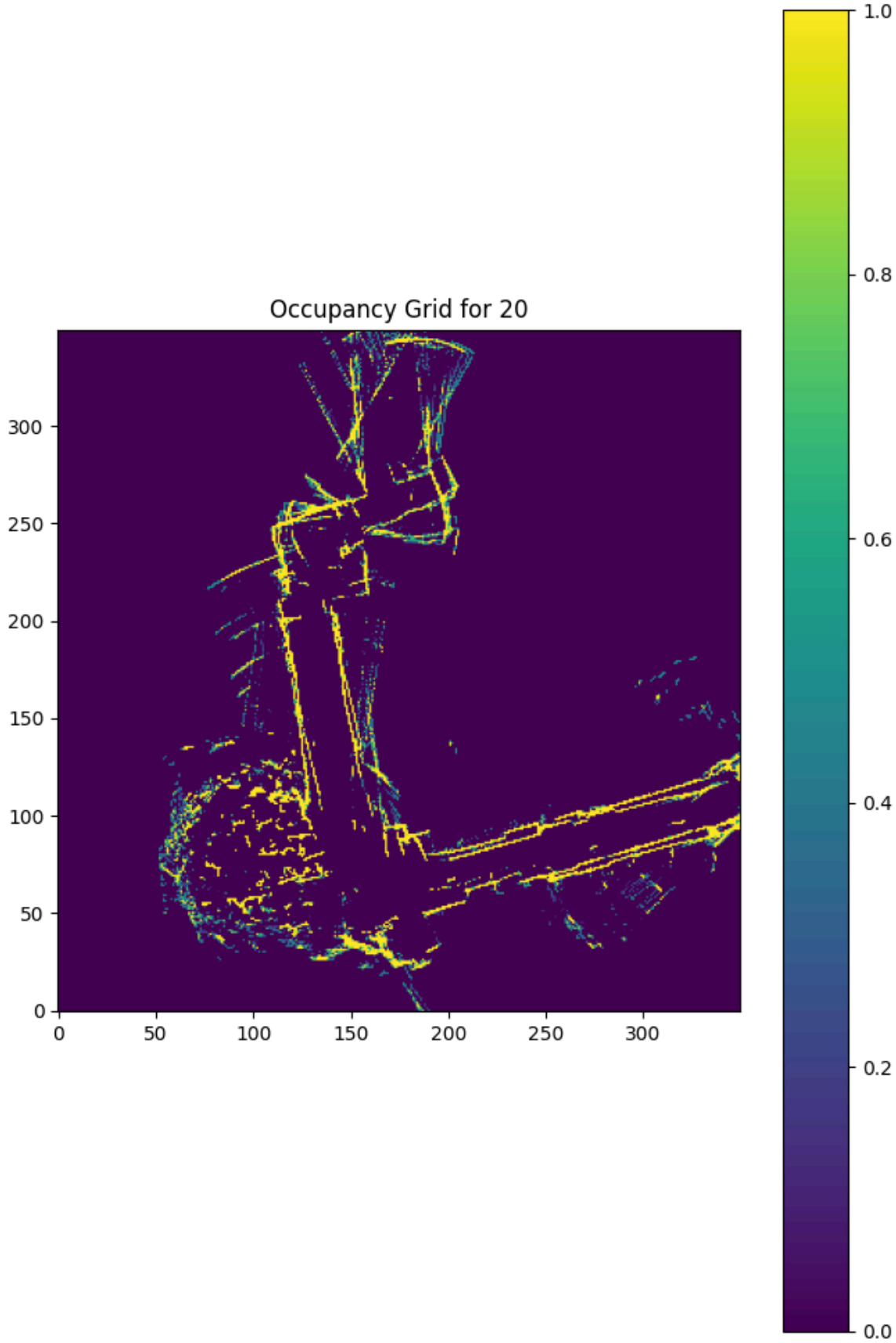

Figure 5 – The occupancy grid constructed of an unknown indoor environemnt, based on the trajectory and observations collected in route 20. A higher value represents a higher log-odds (more likely to be an obstacle/wall).

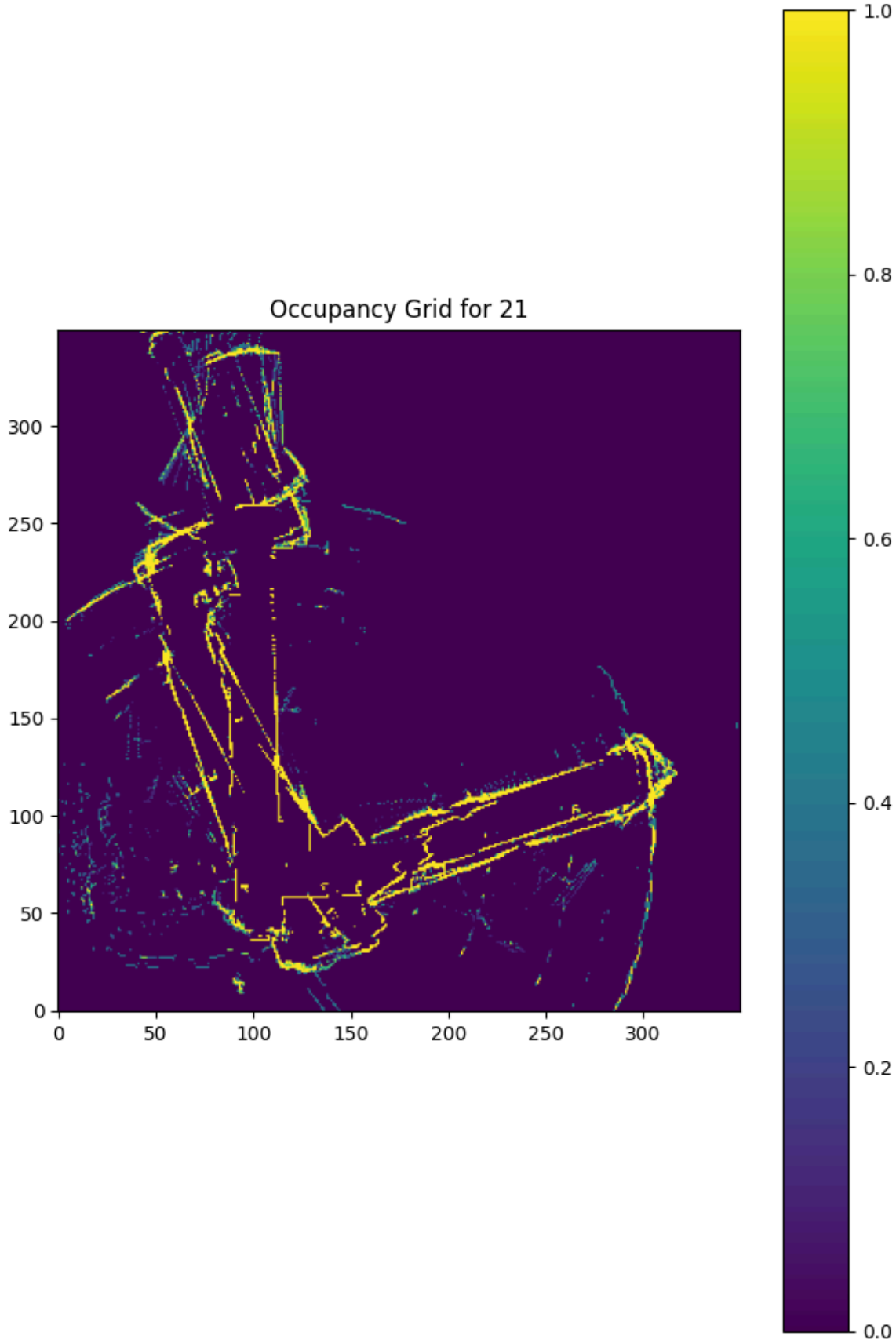

Figure 6 – The occupancy grid constructed of an unknown indoor environemnt, based on the trajectory and observations collected in route 21. A higher value represents a higher log-odds (more likely to be an obstacle/wall).

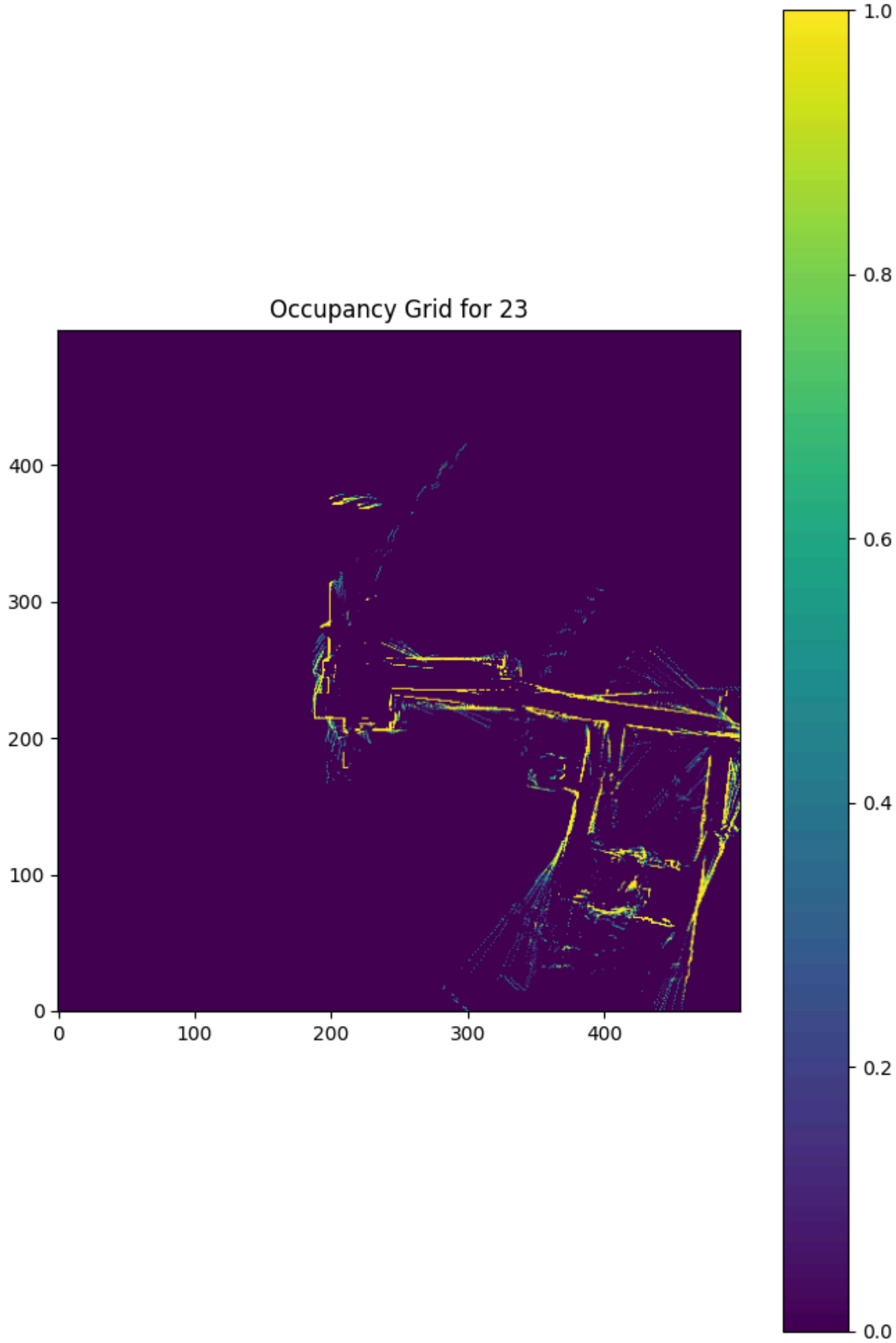

Figure 7 – The occupancy grid constructed of an unknown indoor environemnt, based on the trajectory and observations collected in route 23. A higher value represents a higher log-odds (more likely to be an obstacle/wall).

As we can see, the most uncertainty happens when we are turning, although from Figure 4 we can see that we manage to roughly show the 90 degree turns that the robot takes. We were able to get this result by artificially increasing the body width of our robot chassis in our calculations. Without doing that, we saw our dead reckoning severely overestimating the change in heading direction at each turn.

We also see many faint lines where we might have at some point predicted a hit but we never had any further information to affirm or dissuade that guess, either because the robot turned away or it is at a gridpoint that exceeds the true bounds of the wall (since we only concern ourselves with points between the detected wall/obstacle and the robot, nothing beyond this).

5. Conclusion

Thus far, our mapping and localization results have been separate. In theory, the two can inform each other to help us build a better understanding of the world. Maps tell us where walls are, localization tells us were the robot is. Since a robot cannot be in a wall, we can combine the insight we get from both to make our mapping more precise. This is the guiding principle of simultaneous localization and mapping (SLAM), and we will implement it in the next half of this project.